



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Ingeniería en Inteligencia Artificial



CÓMPUTO PARALELO

Cómputo paralelo utilizando hilos

Práctica 3

Profesor:

José Alfredo Jiménez Benítez

Presentan:

Cruz Trejo Said Carbot

López Castro Magaly

Romero Rosales Lino Ehecatl

Ugalde Téllez Aarón

Grupo: 6BV1

Ciudad de México, a 6 de octubre de 2026

Índice general

Índice de figuras	ii
-------------------	----

Índice de tablas	iii
------------------	-----

1. Cómputo paralelo utilizando hilos	1
1.1. Resumen	1
1.2. Introducción	1
1.3. Marco teórico	2
1.3.1. Cómputo Concurrente y Cómputo Paralelo	2
1.3.2. Arquitectura de Hilos en Sistemas Operativos	3
1.3.3. Concurrencia, Sincronización y Exclusión Mutua	4
1.3.4. Programación de Hilos en C (pthread.h)	4
1.4. Desarrollo del código	4
1.4.1. Diseño de la solución	4
1.4.2. Compilación con Makefile	5
1.4.3. Validaciones	5
1.5. Evidencias de compilación y ejecución	5
1.5.1. Compilación con make	5
1.5.2. Ejecución del programa	6
1.6. Conclusiones	7
1.6.1. Conclusión — Said Carbot Cruz Trejo	8
1.6.2. Conclusión — Magaly López Castro	10
1.6.3. Conclusión — Lino Ehecatl Romero Rosales	11
1.6.4. Conclusión — Aarón Ugalde Téllez	12
1.7. Anexos	14
1.7.1. Código de programa.c	14
1.7.2. Código del Makefile	16

Referencias	17
--------------------	-----------

Índice de figuras

1.1. Captura de la compilación con <code>make</code> en la terminal.	6
1.2. Captura de la ejecución de <code>./programa</code> en la terminal.	7

Índice de tablas

1.1. Distribución de recursos compartidos y privados entre hilos de un mismo proceso.	3
---	---

Práctica 1. Cómputo paralelo utilizando hilos

1.1 Resumen

En esta práctica se desarrolló un programa enfocado en el procesamiento paralelo de estructuras de datos bidimensionales. El proceso principal generó una matriz numérica conformada por tres filas y nueve columnas. Para paralelizar la carga de trabajo, el proceso padre instanció tres hilos independientes, comunicándose con ellos mediante apuntadores para asignar a cada uno una fila específica de la matriz. La tarea designada para cada hilo consistió en procesar los datos recibidos, imprimir su segmento asignado y calcular el producto de los elementos de su respectiva fila. El proceso padre asumió el rol de coordinador, detectando el momento exacto en que los tres hilos terminaron sus respectivos cálculos para, posteriormente, recolectar los resultados alojados en la memoria y mostrarlos en la pantalla, evidenciando el éxito en la comunicación y distribución del procesamiento.

1.2 Introducción

En la actualidad, el procesamiento de grandes volúmenes de datos y la ejecución de cálculos complejos requieren enfoques más eficientes que los proporcionados por la programación secuencial tradicional. El cómputo paralelo surge como una solución fundamental ante las limitaciones físicas de las frecuencias de reloj en los procesadores modernos, permitiendo la ejecución simultánea de múltiples tareas para reducir significativamente los tiempos de procesamiento. Dentro de los sistemas operativos modernos, particularmente en entornos Unix/Linux, una de las herramientas más potentes para implementar este paradigma es el uso de hilos de ejecución (threads) [1].

A diferencia de los procesos tradicionales, que poseen su propio espacio de direcciones de memoria de manera aislada, los hilos son entidades más ligeras que comparten el mismo espacio de memoria, la sección de datos y los recursos del sistema operativo asignados al proceso padre que los originó. Esta característica arquitectónica resulta sumamente ventajosa, ya que reduce la sobrecarga (overhead) computacional inherente a la creación de tareas y facilita enormemente la comunicación entre ellas. Al residir en la misma memoria de acceso aleatorio (RAM), distintos hilos pueden acceder y modificar estructuras de datos compartidas de forma directa.

Sin embargo, esta ventaja también introduce desafíos significativos en el diseño de software. La concurrencia exige mecanismos precisos de coordinación y comunicación para evitar condiciones de carrera y garantizar la integridad de los datos. En el lenguaje de programación C, esta comunicación entre el proceso padre y sus hilos derivados se gestiona frecuentemente a través de apuntadores. El uso de apuntadores permite que el proceso principal asigne dinámicamente bloques de memoria para los datos de entrada e instruya a cada hilo exactamente sobre qué fragmento de información debe operar, recibiendo posteriormente los resultados en direcciones de memoria predefinidas sin necesidad de realizar copias redundantes de la información [1].

En este contexto teórico y práctico, el presente trabajo tiene como **objetivo general** desarrollar programas en lenguaje C que permitan la creación de hilos y la correcta comunicación entre ellos. De manera más granular, el **objetivo específico** radica en establecer un canal de comunicación robusto entre los hilos de trabajo (worker threads) y el proceso padre utilizando apuntadores, validando así la capacidad de dividir una carga de trabajo matemática y consolidar sus resultados.

Para ilustrar y poner en práctica estos conceptos, el problema a resolver consiste en el procesamiento paralelo de una estructura matricial específica. El proceso padre es responsable de inicializar y alojar en memoria RAM la siguiente matriz de orden 3×9 :

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 1 & 3 & 5 & 7 & 9 & 11 & 13 & 15 & 17 \\ 2 & 4 & 6 & 8 & 10 & 12 & 14 & 16 & 18 \end{pmatrix}$$

Una vez estructurados los datos, la estrategia de paralelización dictamina que el proceso padre debe crear tres hilos independientes. Cada uno de estos hilos asume la responsabilidad de procesar una fila completa de la matriz, reportando su identificador único al sistema. La carga de trabajo asignada a cada hilo consiste en realizar la multiplicación secuencial de todos los elementos contenidos en su fila respectiva.

La fase crítica de la práctica ocurre tras el procesamiento: el sistema debe garantizar que el proceso padre tenga la capacidad de detectar cuándo han concluido su labor todos los hilos hijos. Solamente cuando el último hilo ha finalizado su cálculo, el proceso padre debe proceder a leer los resultados obtenidos a través de los apuntadores compartidos y mostrarlos de forma estructurada en pantalla. Este flujo de trabajo no solo demuestra la eficiencia del cómputo paralelo para dividir tareas aritméticas, sino que también subraya la importancia de la sincronización y la gestión de memoria compartida en la arquitectura de sistemas operativos Linux.

1.3 Marco teórico

El guion de la práctica pide investigar el tema de los hilos en sistemas operativos y su programación en lenguaje C. Las subsecciones siguientes desarrollan la distinción entre concurrencia y paralelismo, la arquitectura de hilos, la sincronización y la biblioteca `pthread.h`.

1.3.1 Cómputo Concurrente y Cómputo Paralelo

Al abordar el desarrollo de software multihilo, es necesario distinguir entre concurrencia y paralelismo:

- **Concurrencia:** Propiedad de un sistema para gestionar múltiples tareas intercalando su ejecución en lapsos de tiempo superpuestos. En procesadores de un solo núcleo, esto se logra mediante la alternancia rápida de contextos (*time-slicing*) coordinada por el planificador del sistema operativo.
- **Paralelismo:** Ejecución simultánea de dos o más instrucciones en el mismo instante. Requiere soporte explícito de hardware, como procesadores multinúcleo (*multi-core*) o sistemas multiprocesador.

El límite teórico del incremento de rendimiento al paralelizar un algoritmo está acotado por la Ley de

Amdahl [2]:

$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}}$$

Donde $S(n)$ representa la ganancia en velocidad (*speedup*), p es la fracción del código que se puede ejecutar en paralelo y n es la cantidad de núcleos o hilos de procesamiento disponibles.

1.3.2 Arquitectura de Hilos en Sistemas Operativos

Un proceso es la unidad básica de asignación de recursos administrada por el sistema operativo [3]. Cada proceso posee un espacio de direccionamiento de memoria virtual aislado, tabla de descriptores de archivos, registros de CPU y su propio Bloque de Control de Proceso (PCB). La creación y el cambio de contexto (*context switch*) entre procesos conllevan una sobrecarga relevante debido al intercambio de tablas de páginas y mapas de memoria.

En contraste, un hilo (*thread*) es la unidad mínima de ejecución asignable al procesador. Los hilos creados dentro de un mismo proceso comparten el espacio de direccionamiento global, permitiendo una comunicación más rápida y reduciendo los costos de creación y conmutación.

Recurso / Componente	Compartido	Privado por Hilo
Código del programa (.text)	Sí	No
Variables globales y estáticas (.data)	Sí	No
Memoria dinámica (<i>Heap</i>)	Sí	No
Descriptores de archivos y sockets	Sí	No
Pila de llamadas (<i>Stack</i>) y variables locales	No	Sí
Contador de Programa (PC) y Registros CPU	No	Sí
Identificador de Hilo (TID)	No	Sí

Tabla 1.1. Distribución de recursos compartidos y privados entre hilos de un mismo proceso.

Modelos de Administración de Hilos

1. **Modelo Muchos a Uno ($N : 1$):** Varios hilos en espacio de usuario se asignan a un solo hilo de kernel. Si un hilo ejecuta una llamada al sistema bloqueante, se detiene el proceso completo. No aprovecha arquitecturas multinúcleo.
2. **Modelo Uno a Uno ($1 : 1$):** Cada hilo de usuario se mapea con un hilo del kernel. Ofrece paralelismo real y permite que otros hilos continúen ejecutándose si uno se bloquea. Es el esquema por defecto en sistemas Linux a través de NPTL (*Native POSIX Thread Library*).
3. **Modelo Muchos a Muchos ($M : N$):** Multiplexa M hilos de usuario sobre N hilos de kernel ($M \geq N$), combinando la baja sobrecarga en usuario con la distribución de carga en el kernel.

1.3.3 Concurrencia, Sincronización y Exclusión Mutua

Dado que los hilos comparten el *heap* y la memoria global, el acceso simultáneo a variables compartidas puede provocar condiciones de carrera (*race conditions*), haciendo que el resultado final varíe según el orden de planificación de la CPU.

Para proteger la sección crítica (el segmento de código donde se lee o modifica un recurso compartido), se requiere implementar exclusión mutua:

- **Cerrosos Mutex:** Variables de tipo `pthread_mutex_t` que funcionan como bloqueos binarios para asegurar que un solo hilo acceda a la sección crítica a la vez.
- **Variables de Condición:** Mecanismos que permiten suspender la ejecución de un hilo hasta que reciba una señal de notificación (`pthread_cond_wait`, `pthread_cond_signal`).
- **Semáforos:** Estructuras basadas en contadores enteros que regulan el acceso concurrente a un número determinado de recursos.

El uso incorrecto de estos mecanismos puede originar interbloqueos (*deadlocks*), donde los hilos quedan suspendidos indefinidamente esperando recursos que se retienen mutuamente, o inanición (*starvation*), cuando un hilo no logra acceder al recurso por la prioridad de otros.

1.3.4 Programación de Hilos en C (`pthread.h`)

En sistemas POSIX como Linux, la interfaz estándar para gestionar hilos es la biblioteca `pthread.h` [4]. Las funciones principales utilizadas son:

- `pthread_create()`: Instancia e inicia un nuevo hilo ejecutando la función pasada como argumento mediante un puntero `void*`.
- `pthread_join()`: Bloquea la ejecución del hilo principal hasta que el hilo especificado termine, recolectando su valor de retorno y liberando sus recursos.
- `pthread_exit()`: Termina la ejecución del hilo de forma limpia.
- `pthread_mutex_lock()` / `pthread_mutex_unlock()`: Delimitan la entrada y salida de las secciones críticas.

1.4 Desarrollo del código

1.4.1 Diseño de la solución

El proceso padre (el `main` del programa) crea la matriz de 3×9 en su propia memoria y lanza tres hilos con `pthread_create()`, uno por fila. Como la función de un hilo sólo recibe un apuntador `void *`, el padre empaqueta lo que cada hilo necesita en una estructura `DatosHilo` y le pasa su dirección:

- `numero`: el número de hilo (1, 2 o 3), usado al presentarse en pantalla.
- `fila`: apuntador a la fila de la matriz que le toca (`matriz[i]`); el hilo lee los datos directamente de la memoria del padre, sin copiarlos.
- `columnas`: cantidad de elementos de la fila.
- `resultado`: apuntador a la celda `resultados[i]` del padre, donde el hilo deja su producto.

Cada hilo se presenta con su identificador (`pthread_self()`), muestra los datos que va a procesar, multiplica los elementos de su fila, escribe el producto a través del apuntador `resultado` y muestra el resultado obtenido. Se imprime con una sola llamada a `printf()` por bloque para que las líneas de distintos hilos no se mezclen.

El padre se entera de que los hilos terminaron llamando a `pthread_join()` sobre cada uno: la llamada bloquea al padre hasta que el hilo concluye. Sólo después de los tres `join` lee el arreglo `resultados` y lo muestra. Cada hilo escribe en una celda distinta y sólo lee su propia fila, por lo que no hay variables compartidas modificadas por dos hilos y no se necesitan *mutex*.

Los productos se almacenan en `long long`: el de la tercera fila es de 185 794 560 y crecería rápido si la matriz aumentara de tamaño.

1.4.2 Compilación con `Makefile`

El `Makefile` compila `programa.c` con `gcc`. Usa las banderas `-Wall`, `-Wextra` y `-std=c11`, además de `-pthread`, que enlaza la biblioteca de hilos. Incluye los objetivos `all`, `run` y `clean`. Su código completo está en el anexo.

1.4.3 Validaciones

El programa no solicita datos al usuario (la matriz es fija), por lo que no hay campos que validar. La validación se concentra en revisar el valor de retorno de `pthread_create()` y de `pthread_join()`: si alguno falla, se informa por `stderr` y el programa termina con `EXIT_FAILURE`. No se usan *sockets* ni gestores de bases de datos.

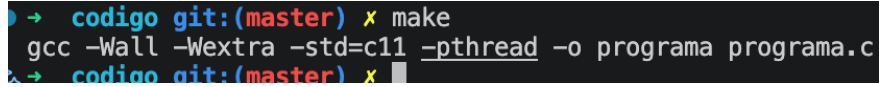
1.5 Evidencias de compilación y ejecución

1.5.1 Compilación con `make`

La compilación termina sin errores ni advertencias.

```
gcc -Wall -Wextra -std=c11 -pthread -o programa programa.c
```

Código 1.1. Compilación con `make`.



```
➔ codigo git:(master) x make
gcc -Wall -Wextra -std=c11 -pthread -o programa programa.c
➔ codigo git:(master) x
```

Figura 1.1. Captura de la compilación con make en la terminal.

1.5.2 Ejecución del programa

Cada hilo se presenta con su identificador, muestra los datos de su fila y su resultado; al final el padre muestra los resultados que leyó tras los `pthread_join()`. Los identificadores cambian en cada ejecución, y el orden de las líneas de los hilos lo decide el planificador.

```
[PADRE] Matriz creada (3 x 9):
1  2  3  4  5  6  7  8  9
1  3  5  7  9 11 13 15 17
2  4  6  8 10 12 14 16 18

[PADRE] Creando 3 hilos...
[HILO 1 | id=6159593472] Datos de la fila: 1 2 3 4 5 6 7 8 9
[HILO 1 | id=6159593472] Resultado (producto de la fila): 362880
[HILO 2 | id=6160166912] Datos de la fila: 1 3 5 7 9 11 13 15 17
[HILO 2 | id=6160166912] Resultado (producto de la fila): 34459425
[HILO 3 | id=6160740352] Datos de la fila: 2 4 6 8 10 12 14 16 18
[HILO 3 | id=6160740352] Resultado (producto de la fila): 185794560

[PADRE] Todos los hilos terminaron. Resultados:
[PADRE] Fila 1 -> 362880
[PADRE] Fila 2 -> 34459425
[PADRE] Fila 3 -> 185794560
```

Código 1.2. Ejecución de `./programa`.

```
➔ código git:(master) x ./programa
[PADRE] Matriz creada (3 x 9):
  1  2  3  4  5  6  7  8  9
  1  3  5  7  9 11 13 15 17
  2  4  6  8 10 12 14 16 18

[PADRE] Creando 3 hilos...
[HILO 1 | id=6162444288] Datos de la fila: 1 2 3 4 5 6 7 8 9
[HILO 1 | id=6162444288] Resultado (producto de la fila): 362880
[HILO 2 | id=6163017728] Datos de la fila: 1 3 5 7 9 11 13 15 17
[HILO 2 | id=6163017728] Resultado (producto de la fila): 34459425
[HILO 3 | id=6163591168] Datos de la fila: 2 4 6 8 10 12 14 16 18
[HILO 3 | id=6163591168] Resultado (producto de la fila): 185794560

[PADRE] Todos los hilos terminaron. Resultados:
[PADRE] Fila 1 -> 362880
[PADRE] Fila 2 -> 34459425
[PADRE] Fila 3 -> 185794560
➔ código git:(master) x
```

Figura 1.2. Captura de la ejecución de `./programa` en la terminal.

Los productos se comprueban a mano: la fila 1 es $9! = 362\,880$; la fila 2 es el producto de los impares de 1 a 17, $34\,459\,425$; y la fila 3 es $2^9 \cdot 9! = 185\,794\,560$.

1.6 Conclusiones

A continuación cada integrante del equipo comparte su experiencia individual con el desarrollo de esta práctica.

1.6.1 Conclusión — Said Carbot Cruz Trejo

Durante la realización de esta práctica y la investigación previa sobre sistemas operativos y la gestión de procesos, pude comprender a profundidad la vital importancia del cómputo paralelo en la actualidad. Anteriormente, la concepción de la programación se limitaba a un enfoque estrictamente secuencial, donde cada instrucción debía esperar pacientemente a que la anterior concluyera su ciclo en el procesador. Sin embargo, al adentrarme en el desarrollo con la biblioteca de hilos en lenguaje C, experimenté un cambio de paradigma significativo. La investigación bibliográfica me permitió entender teóricamente qué es un hilo y cómo se diferencia fundamentalmente de un proceso completo, destacando que los hilos comparten el mismo espacio de memoria y descriptores de archivos, lo cual reduce drásticamente la sobrecarga de creación y el cambio de contexto por parte del sistema operativo subyacente. Esto se comprobó empíricamente al compilar y ejecutar los códigos de prueba, donde la instanciación de múltiples hilos desvinculados se realizó de manera fluida y con un consumo mínimo de recursos computacionales.

Al analizar la comunicación entre el proceso padre y los hilos mediante el uso de apuntadores, corroboré la naturaleza de la memoria compartida que detalla la literatura técnica. En uno de los ejercicios desarrollados, se enviaron argumentos a un hilo a través de un arreglo de enteros convertido a un apuntador genérico. Resultó fascinante observar cómo las modificaciones realizadas por el hilo sobre esos valores de memoria específicos se reflejaban de inmediato en el proceso padre una vez que el hilo terminaba su ejecución. Esta característica, aunque inmensamente poderosa para el paso de información, también me hizo reflexionar sobre los peligros inherentes de la concurrencia. Si múltiples hilos intentaran modificar la misma dirección de memoria simultáneamente sin los mecanismos de sincronización adecuados, se producirían condiciones de carrera. Aunque en esta práctica los ejercicios estaban estructurados lógicamente para evitar conflictos directos mediante la asignación de filas independientes, la investigación teórica dejó muy claro que el diseño cuidadoso y preventivo es indispensable para garantizar la integridad de los datos en sistemas paralelos reales.

El ejercicio principal, enfocado en el procesamiento de una matriz matemática de tres por nueve, consolidó todos los conceptos teóricos y prácticos abordados durante el periodo de estudio. El proceso padre asumió el rol de coordinador central, encargándose de inicializar los datos y distribuir la carga de trabajo de manera equitativa. Al crear tres hilos distintos y asignar a cada uno una fila específica para realizar la multiplicación continua de sus elementos, se logró una verdadera paralelización de tareas informáticas. Pude visualizar cómo la unidad central de procesamiento asignaba diferentes núcleos a cada hilo según la disponibilidad, permitiendo que las multiplicaciones se ejecutaran de forma simultánea.

En el contexto de la Ingeniería en Inteligencia Artificial, que es el marco principal de esta formación académica, los conocimientos adquiridos en esta práctica trascienden el mero ejercicio de programación. La inteligencia artificial moderna, especialmente el aprendizaje profundo y el entrenamiento de modelos masivos, depende fundamentalmente de la capacidad de paralelizar operaciones algebraicas a gran escala. Las multiplicaciones matriciales son el núcleo computacional de cualquier red neuronal. Entender desde el nivel más bajo cómo un sistema operativo gestiona estos recursos de procesamiento me ha dotado de una base teórica y práctica excepcionalmente sólida para optimizar algoritmos en el futuro. Me di cuenta de que el rendimiento real no solo depende de la complejidad teórica del código, sino de qué tan bien aprovecha la arquitectura física del procesador. La optimización del tiempo de ejecución demostrada en esta práctica es el primer peldaño para desarrollar sistemas inteligentes capaces de procesar información de manera instantánea.

En conclusión, el desarrollo de la investigación y la implementación técnica de la práctica transformaron conceptos abstractos en habilidades tangibles que impactarán directamente en mi perfil profesional.

1.6.2 Conclusión — Magaly López Castro

El desarrollo de esta práctica representó una inmersión profunda en los fundamentos del cómputo concurrente, alejándose de los paradigmas secuenciales para abordar el manejo y la administración de hilos directamente desde el lenguaje C. Al implementar la multiplicación de los elementos de una matriz de 3×9 , dividiendo la carga de trabajo fila por fila entre hilos independientes, se comprueba empíricamente cómo la distribución de tareas optimiza el uso de los recursos.

A nivel técnico, el mayor reto y a la vez el aprendizaje más valioso consistió en orquestar la comunicación entre el proceso padre y los hilos hijos empleando apuntadores. Manipular la memoria compartida en RAM exige una lógica de programación sumamente rigurosa. Es indispensable asegurar que cada hilo acceda exactamente a la dirección de memoria que le corresponde para leer sus datos de entrada y escribir su resultado, evitando el riesgo de sobrescribir información o de generar violaciones de segmento (segmentation faults). Asimismo, la fase de sincronización resultó reveladora; el proceso padre no puede simplemente lanzar los hilos y continuar su camino asumiendo que los datos estarán listos. Requiere mecanismos explícitos para monitorear el estado de los hilos, aguardar a que todos terminen su ejecución y, solo entonces, recolectar los resultados a través de los apuntadores para presentarlos en pantalla.

Más allá de la sintaxis del código, la ejecución de estos programas desde la terminal en un entorno Linux resalta la enorme flexibilidad y robustez que ofrecen los sistemas basados en Unix para el desarrollo a bajo nivel. Interactuar directamente con el planificador del sistema operativo permite comprender qué es lo que realmente sucede detrás de las abstracciones modernas. En un ámbito donde se persigue la eficiencia algorítmica y donde técnicas como la optimización de procesos o incluso las búsquedas de fuerza bruta requieren explotar al máximo el hardware disponible, entender cómo paralelizar las instrucciones de un programa representa una ventaja técnica sustancial. Modificar un algoritmo para que aproveche arquitecturas concurrentes no solo mejora los tiempos de ejecución, sino que transforma por completo la manera en la que se modela un problema.

Dentro de la formación en Ingeniería en Inteligencia Artificial, esta práctica adquiere un nivel de relevancia crítico. La gran mayoría de los modelos modernos, desde la extracción de características en técnicas de procesamiento de lenguaje natural hasta el entrenamiento de redes neuronales y algoritmos bioinspirados, dependen intrínsecamente del cómputo paralelo. Las cargas de trabajo que implican enormes matrices y grandes volúmenes de información superan rápidamente las capacidades de la programación secuencial.

Comprender desde ahora, a nivel de lenguaje C, cómo se instancian los hilos, cómo comparten un mismo espacio de memoria y cómo sincronizan sus ciclos de vida, es sentar los cimientos tecnológicos necesarios para el futuro. Este entendimiento será vital al escalar proyectos y al interactuar más adelante con arquitecturas avanzadas, supercómputo y frameworks de procesamiento paralelo masivo. En síntesis, esta práctica superó la meta inicial de la gestión y operación de una matriz; logró afianzar conceptos vitales sobre arquitectura de computadoras, control seguro de memoria y optimización matemática, pilares que resultan indispensables para el desarrollo de tecnologías inteligentes eficientes.

1.6.3 Conclusión — Lino Ehecatl Romero Rosales

El desarrollo de esta práctica fue un poco mas complejo pues esta se alejaba de algo secuencial para abordar la administración directa de hilos en la que accedimos mediante la biblioteca POSIX Threads en Linux. Al implementar el procesamiento de una matriz, dividiendo la carga de trabajo entre tres hilos independientes para que cada uno calculara el producto acumulado de los elementos de su fila, se comprobó cómo la descomposición de tareas permite explotar la memoria compartida de manera eficiente.

El reto central consistio en orquestar la comunicación entre el hilo padre y los hilos hijos utilizando únicamente punteros. Debido a que la rutina de instanciación recibe un puntero de tipo vacío, fue necesario empaquetar la dirección en memoria de cada fila, la cantidad de columnas y el identificador de cada hilo dentro de una estructura de datos. Al realizar la conversión de tipo al inicio de la ejecución de cada hilo hijo, se garantizó un acceso a los datos sin duplicar memoria. Manipular el espacio de direccionamiento global exige una lógica estricta para asegurar que cada hilo lea y escriba en las direcciones correctas, evitando condiciones de carrera. Asimismo cuando llegamos a la fase de sincronización nos pudo mostrar la necesidad de llamadas explícitas usando `pthread_join`, para así garantizarnos que el proceso padre recolectara los resultados consolidados de los tres productos únicamente cuando todos los hilos hubieran concluido su ejecución.

Más allá de la sintaxis y de las reglas del proyecto —como la automatización de la compilación mediante usando `Makefile`, trabajar directamente sobre la terminal de Linux resalta la robustez de los sistemas POSIX para el desarrollo de procesos en paralelo. Comprender cómo interactúa el código con el planificador del sistema operativo permite asimilar los mecanismos reales que llegan a asimilarse a las abstracciones modernas. En puntos donde el procesamiento en paralelo es mas eficiente para saber paralelizar instrucciones representa un gran aprendizaje fundamental que transforma la manera en la que se diseñan los sistemas.

Esto ayudandonos en el aprendizaje para el avanve de la Inteligencia Artificial, esta experiencia es un gran aprendizaje para la implementacion de procesos paralelos en la IA. El entrenamiento de redes neuronales, la extracción de características en el procesamiento de lenguaje natural y los algoritmos bioinspirados se basan en el uso de operaciones matriciales y procesamiento paralelo. El empezar a comprender cómo se instancian los hilos, cómo comparten memoria de forma segura y cómo se sincronizan sus ciclos de vida constituye la base para emepezar a hacer la creacion de arquitecturas mas complejas como podria ser trabajar con computadoras o servidores mas complejos tanto como llegar a optimizar las cargas de trabajo que exigen las tecnologías inteligentes del futuro.

1.6.4 Conclusión — Aarón Ugalde Téllez

En la práctica anterior repartimos el trabajo entre procesos creados con `fork()`, y la mayor dificultad fue que cada hijo recibe una copia independiente de la memoria, así que los resultados tuvieron que viajar por un archivo. Con hilos el problema se invierte: todos comparten el espacio de direcciones del proceso, de modo que la comunicación con el padre puede resolverse con apuntadores y sin archivos intermedios. Esa diferencia fue lo que más me llamó la atención al implementar la práctica, porque el mismo problema de la matriz de tres filas por nueve columnas quedó resuelto con mucho menos código y sin ningún mecanismo externo de intercambio de datos.

El diseño giró en torno a un detalle propio de `pthread_create()`: la función del hilo sólo recibe un apuntador `void *`, así que no se le pueden pasar varios argumentos por separado. La solución fue definir una estructura `DatosHilo` con el número de hilo, un apuntador a la fila que le toca, la cantidad de columnas y un apuntador a la celda del arreglo de resultados donde debe escribir. El padre llena una estructura por hilo y pasa su dirección; el hilo la convierte de nuevo a su tipo y trabaja directamente sobre la memoria del padre, sin copiar la fila. Entendí que esas estructuras deben vivir en el `main` (en un arreglo) y no como variables temporales dentro del ciclo, porque el hilo las lee cuando el planificador decide ejecutarlo, y si la variable ya hubiera desaparecido se leería memoria inválida. Por eso el arreglo `datos` y el arreglo `resultados` se declaran antes de crear los hilos y siguen existiendo hasta que todos terminan.

Otro punto importante fue la sincronización. El padre no puede leer los resultados apenas lanza los hilos, porque quizá ninguno haya terminado todavía y el arreglo seguiría con sus valores iniciales. Llamar a `pthread_join()` sobre cada hilo resuelve las dos cosas a la vez: bloquea al padre hasta que ese hilo concluye y garantiza que lo que el hilo escribió ya es visible cuando el padre lo lee. En la ejecución se ve con claridad: las líneas de los hilos aparecen primero y el bloque de resultados del padre aparece al final, con los tres productos correctos. También noté que el orden en que los hilos imprimen lo decide el planificador del sistema operativo y no el programa, y que los identificadores cambian en cada ejecución. Para que las líneas de un mismo hilo no se mezclaran con las de otro, cada hilo arma su mensaje en un búfer y lo imprime con una sola llamada a `printf()`.

Sobre la concurrencia, en esta práctica no hizo falta usar un *mutex*, y creo que entender por qué es parte del aprendizaje: cada hilo lee una fila distinta y escribe en una celda distinta, por lo que ningún dato es modificado por dos hilos a la vez y no existe una condición de carrera. Si en cambio los tres hilos hubieran acumulado su producto en una misma variable, el resultado habría dependido del orden de ejecución y habría sido necesario proteger esa sección crítica. Por la misma razón de prudencia usé `long long` para los productos: el de la tercera fila ya llega a 185 794 560 y crecería muy rápido si la matriz fuera más grande.

La práctica no solicita datos al usuario, así que la validación se concentró en revisar el valor de retorno de `pthread_create()` y de `pthread_join()`, informando por `stderr` y terminando con `EXIT_FAILURE` si alguna falla. El Makefile con `-Wall`, `-Wextra` y `-std=c11` me obligó a entregar código sin advertencias, y la bandera `-pthread` fue necesaria para enlazar la biblioteca de hilos; sin ella la compilación no termina.

Para mi formación en Inteligencia Artificial, lo más valioso es ver que repartir una operación sobre una matriz entre varios flujos de ejecución es, en pequeño, lo que hacen las bibliotecas de álgebra lineal detrás del entrenamiento de redes neuronales. Aquí el trabajo de cada hilo es mínimo y el costo de crearlo supera a lo

que ahorra, así que no se espera una ganancia de tiempo; el objetivo era entender el mecanismo. Con él claro, estoy mejor preparado para los temas siguientes, donde el reparto de trabajo ya sí tendrá que justificarse con mediciones de desempeño.

1.7 Anexos

Se incluyen los códigos generados para esta práctica.

1.7.1 Código de programa.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4
5 #define FILAS 3
6 #define COLUMNAS 9
7 #define NUM_HILOS FILAS
8
9 /* Datos que el proceso padre le pasa a cada hilo mediante un apuntador.
10  * - fila: apuntador a la fila de la matriz que debe procesar.
11  * - resultado: apuntador a la celda del arreglo de resultados donde el
12  *           hilo escribe el producto (memoria del proceso padre). */
13 typedef struct {
14     int numero;
15     const int *fila;
16     int columnas;
17     long long *resultado;
18 } DatosHilo;
19
20 /* Funcion que ejecuta cada hilo: multiplica los elementos de su fila. */
21 static void *multiplicar_fila(void *arg) {
22     DatosHilo *datos = (DatosHilo *)arg;
23     char linea[256];
24     int pos = 0;
25     long long producto = 1;
26
27     pos += snprintf(linea + pos, sizeof(linea) - (size_t)pos,
28                     "[HILO %d | id=%lu] Datos de la fila:",
29                     datos->numero, (unsigned long)pthread_self());
30
31     for (int j = 0; j < datos->columnas; j++) {
32         pos += snprintf(linea + pos, sizeof(linea) - (size_t)pos,
33                         " %d", datos->fila[j]);
34         producto *= datos->fila[j];
35     }
36
37     /* Escritura del resultado en la memoria del proceso padre. */
38     *(datos->resultado) = producto;
39
40     /* Un solo printf por hilo para que las lineas no se mezclen. */
```

```
41     printf("%s\n[HILO %d | id=%lu] Resultado (producto de la fila): %lld\n",
42           linea, datos->numero, (unsigned long)pthread_self(), producto);
43
44     return NULL;
45 }
46
47 int main(void) {
48     int matriz[FILAS][COLUMNAS] = {
49         {1, 2, 3, 4, 5, 6, 7, 8, 9},
50         {1, 3, 5, 7, 9, 11, 13, 15, 17},
51         {2, 4, 6, 8, 10, 12, 14, 16, 18}
52     };
53     long long resultados[NUM_HILOS] = {0};
54     pthread_t hilos[NUM_HILOS];
55     DatosHilo datos[NUM_HILOS];
56
57     printf("[PADRE] Matriz creada (%d x %d):\n", FILAS, COLUMNAS);
58     for (int i = 0; i < FILAS; i++) {
59         for (int j = 0; j < COLUMNAS; j++) {
60             printf("%3d ", matriz[i][j]);
61         }
62         printf("\n");
63     }
64
65     printf("\n[PADRE] Creando %d hilos...\n", NUM_HILOS);
66     for (int i = 0; i < NUM_HILOS; i++) {
67         datos[i].numero = i + 1;
68         datos[i].fila = matriz[i];
69         datos[i].columnas = COLUMNAS;
70         datos[i].resultado = &resultados[i];
71
72         int rc = pthread_create(&hilos[i], NULL, multiplicar_fila, &datos[i]);
73         if (rc != 0) {
74             fprintf(stderr, "Error al crear el hilo %d (codigo %d)\n", i + 1,
75                     rc);
76             return EXIT_FAILURE;
77         }
78
79         /* El padre detecta que los hilos terminaron esperandolos con join. */
80         for (int i = 0; i < NUM_HILOS; i++) {
81             int rc = pthread_join(hilos[i], NULL);
82             if (rc != 0) {
83                 fprintf(stderr, "Error al esperar al hilo %d (codigo %d)\n", i + 1,
84                         rc);
85                 return EXIT_FAILURE;
86             }
87         }
88     }
```

```
86     }
87
88     printf("\n[PADRE] Todos los hilos terminaron. Resultados:\n");
89     for (int i = 0; i < NUM_HILOS; i++) {
90         printf("[PADRE] Fila %d -> %lld\n", i + 1, resultados[i]);
91     }
92
93     return EXIT_SUCCESS;
94 }
```

Código 1.3. Proceso padre e hilos que multiplican las filas de la matriz (programa.c).

1.7.2 Código del Makefile

```
1 CC = gcc
2 CFLAGS = -Wall -Wextra -std=c11 -pthread
3
4 TARGET = programa
5
6 all: $(TARGET)
7
8 $(TARGET): programa.c
9     $(CC) $(CFLAGS) -o $(TARGET) programa.c
10
11 run: $(TARGET)
12     ./$(TARGET)
13
14 clean:
15     rm -f $(TARGET) a.out
16
17 .PHONY: all run clean
```

Código 1.4. Makefile para compilar el programa.

Referencias

- [1] F. Márquez, *Unix: Programación Avanzada*. Grupo Editorial Ra-Ma, 2015.
- [2] G. M. Amdahl, «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,» en *Proceedings of the AFIPS Spring Joint Computer Conference*, Atlantic City, NJ, USA, 1967, págs. 483-485.
- [3] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ: Wiley, 2018.
- [4] W. R. Stevens y S. A. Rago, *Advanced Programming in the UNIX Environment*, 3.^a ed. Addison-Wesley, 2013.